Problem Set 1: Threaded Applications

Abraham Sharum

Due Date: October 3, 2025
Class: CSE 30003 - Distributed Systems

October 6, 2025

Deliverables

PS1.java

```
/*******************************
Name: Abraham Sharum
Problem Set: PS1
Due Date: October 3, 2025
*******************************/

import java.io.*;
import java.util.*;

public class PS1 {
    public static void main(String[] args) {
        int totalSum = 0;
        int numThreads = 0;
        String file = "";
        ArrayList<Integer> temp = new ArrayList<>();
        int[] nums = new int[0];
        Thread[] threads = new Thread[0];
        ArraySum[] runners = new ArraySum[0];


        try {
            if (args.length != 2) throw new IllegalArgumentException("Need two arguments");
            file = args[0];
            numThreads = Integer.parseInt(args[1]);
            threads = new Thread[numThreads];
            runners = new ArraySum[numThreads];
        } catch (IllegalArgumentException e) {
            e.printStackTrace();
            System.exit(0);
        }

        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while((line = br.readLine()) != null) temp.add(Integer.parseInt(line));
            nums = new int[temp.size()];
            for (int i = 0; i < nums.length; i++) nums[i] = temp.get(i);
            temp.clear();

        } catch (IOException e) {
            e.printStackTrace();
            System.exit(0);
        } catch (NumberFormatException e) {
            e.printStackTrace();
            System.exit(0);
        }
```

```
int startIndex = 0;
int threadIndex = 0;
int range = nums.length / numThreads;
int leftOver = nums.length > numThreads ? nums.length % numThreads : 0;

System.out.printf("Number of threads : %d\n",numThreads);
System.out.printf("Number of elements: %d\n",nums.length);
System.out.printf("Number of leftover elements: %d \nThe first: %d threads are getting an
    additonal element.\n",leftOver,leftOver);
System.out.println("=".repeat(54) + "\n");

try {

    if (numThreads <= nums.length) {

        while (threadIndex < numThreads) {
            // First k threads get an extra element until the number of leftover elements
                is reached
            int extendRange = threadIndex < leftOver ? 1 : 0;
            int endIndex = startIndex + range + extendRange - 1;

            runners[threadIndex] = new ArraySum(startIndex, endIndex, nums,
                threadIndex);
            threads[threadIndex] = new Thread(runners[threadIndex]);
            threads[threadIndex++].start();
            startIndex = endIndex + 1;
        }
    } else {
        for (int i = 0; i < numThreads; i++) {
            runners[i] = new ArraySum(i, i, nums,i);
            threads[i] = new Thread(runners[i]);
            threads[i].start();
        }
    }

    for (int i = 0; i < threads.length; i++) threads[i].join();

    for(int i = 0; i < runners.length; i++) {
        ArraySum tempRunner = runners[i];
        System.out.print(tempRunner);
        totalSum += tempRunner.getSum();
    }

    System.out.printf("\nFinal Result: <%d>\n",totalSum);

} catch (InterruptedException e) {
    e.printStackTrace();
    System.exit(0);
```

```
        }
    }
}
```

ArraySum.java

```
/*******************************
Name: Abraham Sharum
Problem Set: PS1
Due Date: October 3, 2025
*******************************/

class ArraySum implements Runnable {
    private int localSum = 0;
    private int[] nums;
    private int startIndex;
    private int endIndex;
    private int threadNum;

    public ArraySum(int startIndex, int endIndex, int[] nums, int threadNum) {
        this.startIndex = startIndex;
        this.endIndex = endIndex;
        this.nums = nums;
        this.threadNum = threadNum + 1;
    }

    public int getSum() {
        return this.localSum;
    }

    public int getStartIndex() {
        return this.startIndex;
    }

    public int getEndIndex() {
        return this.endIndex;
    }

    public void run () {
        if (endIndex >= nums.length) return;
        for (int i = startIndex; i <= endIndex; i++) localSum += nums[i];
    }

    public String toString() {
        if (endIndex >= nums.length) return String.format("Thread-<%d>:-Invalid-index-range-
            of-range-<%d>-and-<%d>.\n",this.threadNum,this.startIndex,this.endIndex);
        return String.format("Thread-<%d>:-Partial-sum-for-range-<%d>/<%d>-is-%d.\n",
            this.threadNum,this.startIndex,this.endIndex,this.localSum);
    }
}
```